THE NATIONAL UNIVERSITY OF LESOTHO

Department of Mathematics and Computer Science

CS3520 - Computer Organisation and Architecture I

Tutorial 1

15th September 2025

1. List the four basic elements of a computer system and briefly explain the role of each component.
2. Describe the steps that transform a program written in a high-level language such as C into a representation that is directly executed by a computer processor.
3. Consider three different processors P1, P2, and P3 executing the same instruction set. P1 has a 3 GHz clock rate and a CPI of 1.5. P2 has a 2.5 GHz clock rate and a CPI of 1.0. P3 has a 4.0 GHz clock rate and has a CPI of 2.2.
   (a) Which processor has the highest performance expressed in instructions per second?
   (b) If the processors each execute a program in 10 seconds, find the number of cycles and the number of instructions.
   (c) We are trying to reduce the execution time by 30%, but this leads to an increase of 20% in the CPI. What clock rate should we have to get this time reduction?
4. Processor registers are classified into two broad categories. Name these categories and describe their roles.
5. The user-visible registers are further subdivided into three types. Identify these types and explain the purpose of each.
6. Describe the steps in the basic instruction cycle. What roles do the program counter (PC) and instruction register (IR) play during this cycle?
7. According to the computer system overview, instructions fall into four categories. Name and briefly describe each category.
8. Consider the RISC-V load instructions LB, LH, LW, LBU and LHU. Which of these instructions sign-extend the loaded value, and which zero-extend it? What sizes of data do they load?
9. Explain how the RISC-V store instructions SB, SH and SW operate. What size of data does each store, and from which register is the data taken?
10. Why does the RISC-V ISA recommend that effective addresses for loads and stores be naturally aligned? What may happen when accesses are misaligned?
11. Describe the purpose of ADDI, SLTI and SLTIU instructions. How do immediate instructions differ from their register-register counterparts such as ADD and SLT?
12. Describe how the ADD, SUB and SLT instructions work in the RISC-V ISA. Which registers do they read and write?
13. Explain the RISC-V logical instructions AND, OR, XOR and the shift instructions SLL, SRL and SRA. How does an arithmetic right shift (sra/srai) differ from a logical right shift (srl/srli)?

14. What is the canonical NOP instruction in RISC-V? How is it encoded and why does the ISA define a canonical encoding for a no-operation?

15. RISC-V branch instructions compare two registers and alter control flow. Describe how BEQ, BNE, BLT, BLTU, BGE and BGEU work. Why does RISC-V combine comparison and branch into a single instruction instead of using condition codes?

16. Write a RISC-V assembly loop that computes the sum of an integer array. The base address of the array is in x10, the number of elements N is in x11, and the sum should be returned in x12. Use lw to load each element and addi or add to accumulate. Use beq/bne to terminate the loop when the index reaches N.

17. What does it mean that RISC-V is a load–store architecture? Why are arithmetic instructions permitted to operate only on registers, and how does the architecture support memory access?

18. Give an example of a misaligned memory access in RISC-V. What happens when an address is not naturally aligned, and why might hardware support for misaligned accesses be optional?

19. How are JAL (jump and link) and JALR (jump and link register) used for function calls in RISC-V? How is the return address saved, and how is the target address computed?

20. Demonstrate how to push and pop a register value to and from the stack in RISC-V assembly. The stack pointer is in x2 (also called sp). Use addi to adjust the stack pointer and appropriate store/load instructions.

21. Write a RISC-V assembly snippet that implements the following high-level code:

```
if (x5 < x6) {
    x7 = x5;
} else {
    x7 = x6;
}
```

22. For the RISC-V assembly instructions below, what is the corresponding C statement? Assume that the variables f, g, h, i, and j are assigned to registers x5, x6, x7, x28, and x29, respectively. Assume that the base address of the arrays A and B are in registers x10 and x11, respectively.

```
slli x30, x5, 3
add x30, x10, x30
slli x31, x6, 3
add x31, x11, x31
lw x5, 0(x30)
addi x12, x30, 8
lw x30, 0(x12)
add x30, x30, x5
lw x30, 0(x31)
```

23. Show how the value 0xabcdef12 would be arranged in memory of a little-endian and a big-endian machine. Assume the data are stored starting at address 0 and that the word size is 4 bytes.

24. Consider a proposed new instruction named rpt. This instruction combines a loop's condition check and counter decrement into a single instruction. For example rpt x29, loop would do the following:

```
if (x29 > 0) {
    x29 = x29 −1;
    goto loop
}
```

(a) If this instruction were to be added to the RISC-V instruction set, what is the most appropriate instruction format?

(b) What is the shortest sequence of RISC-V instructions that performs the same operation?

25. Consider the following RISC-V loop:

```
LOOP:
    beq x6, x0, DONE
    addi x6, x6, -1
    addi x5, x5, 2
    jal x0, LOOP
DONE:
```

(a) Assume that the register x6 is initialized to the value 10. What is the final value in register x5 assuming the x5 is initially zero?

(b) For the loop above, write the equivalent C code. Assume that the registers x5 and x6 are integers acc and i, respectively.

(c) For the loop written in RISC-V assembly above, assume that the register x6 is initialized to the value N. How many RISC-V instructions are executed?

(d) For the loop written in RISC-V assembly above, replace the instruction "beq x6, x0, DONE" with the instruction "blt x6, x0, DONE" and write the equivalent C code.